

Otimizações de Performance - Clube do Bem

Este documento descreve as otimizações de performance implementadas no projeto para melhorar os Core Web Vitals (LCP, FID, CLS).



Otimizações Implementadas

1. Lazy Loading e Code Splitting

Componentes Lazy Loading

- **UnifiedHeader**: Carregamento dinâmico de UserNav e CartIcon
- **Sidebar**: Lazy loading de componentes pesados do Radix UI
- **ServiceBookingPage**: Dynamic imports para componentes de UI

Utilitários de Lazy Loading

- `src/lib/lazy-components.ts` : Centraliza todos os componentes lazy-loaded
- Componentes administrativos carregados sob demanda
- Componentes de marketplace com lazy loading

2. Memoização com React.memo

Componentes Otimizados

- **UnifiedHeader**: Memoizado com useCallback e useMemo
- **UserNav**: Memoização para prevenir re-renders desnecessários
- **CartIcon**: Memoização com cálculo otimizado do contador
- **ServiceBookingPage**: Memoização da página completa

Hooks de Performance

- `src/hooks/usePerformance.ts` : Hooks personalizados para otimização
- `useDebounce` : Previne re-renders excessivos
- `useThrottle` : Controla frequência de chamadas de função
- `useIntersectionObserver` : Lazy loading baseado em visibilidade

3. Otimização de Imagens

Componente OptimizedImage

- `src/components/OptimizedImage.tsx` : Wrapper otimizado do next/image
- Lazy loading automático
- Placeholder blur personalizado
- Tratamento de erros
- Presets para diferentes casos de uso

Configurações next/image

- Domínios remotos configurados (Supabase, Unsplash)
- Qualidade otimizada por contexto
- Sizes responsivos configurados

4. Code Splitting Avançado

Configuração Webpack

- **Vendor chunk:** Bibliotecas de terceiros separadas
- **UI components chunk:** Componentes de UI agrupados
- **Admin chunk:** Páginas administrativas isoladas
- **Marketplace chunk:** Funcionalidades de marketplace
- **Common chunk:** Componentes compartilhados

Configurações de Bundle

- `minSize: 20000` : Tamanho mínimo para chunks
- `maxAsyncRequests: 30` : Máximo de requests assíncronos
- Tree shaking habilitado em produção

5. Preload de Recursos Críticos

Componente PreloadResources

- `src/components/PreloadResources.tsx` : Gerencia preloads
- DNS prefetch para domínios externos
- Preconnect para origens críticas
- Preload de fontes, imagens e scripts críticos

Configurações por Página

- Home: Hero images e fontes principais
- Marketplace: Imagens de categoria
- Admin: Scripts de dashboard

6. Configurações Experimentais

Next.js Experimental Features

- `optimizeCss: true` : Otimização de CSS
- `scrollRestoration: true` : Restauração otimizada de scroll
- `legacyBrowsers: false` : Remove suporte a browsers antigos
- `forceSwcTransforms: true` : Força uso do SWC

7. Monitoramento de Performance

Hooks de Monitoramento

- `usePerformance` : Monitora Core Web Vitals
- `useRenderPerformance` : Mede tempo de render de componentes
- Métricas coletadas: LCP, FID, CLS, FCP, TTFB

Scripts de Análise

- `npm run build:analyze` : Análise de bundle
- `npm run perf:lighthouse` : Relatório Lighthouse
- `npm run perf:bundle` : Análise de tamanho de bundles



Impacto Esperado nos Core Web Vitals

Largest Contentful Paint (LCP)

-  Preload de recursos críticos

-  Otimização de imagens com next/image
-  Code splitting para reduzir bundle inicial
-  Lazy loading de componentes não críticos

First Input Delay (FID)

-  Code splitting reduz JavaScript inicial
-  Lazy loading de componentes pesados
-  Memoização previne re-renders desnecessários
-  Throttling e debouncing de eventos

Cumulative Layout Shift (CLS)

-  Placeholders para imagens com dimensões fixas
-  Skeleton loaders para componentes lazy
-  Preload de fontes críticas
-  Dimensões explícitas em componentes



Como Usar

Desenvolvimento

```
# Executar em modo desenvolvimento
npm run dev

# Analisar performance durante desenvolvimento
npm run perf:lighthouse
```

Produção

```
# Build otimizado
npm run build

# Análise de bundle
npm run build:analyze

# Relatório de performance
npm run perf:bundle
```

Monitoramento

```
import { usePerformance } from '@/hooks/usePerformance'

function MyComponent() {
  const metrics = usePerformance()

  // Métricas disponíveis: lcp, fid, cls, fcp, ttfb
  console.log('LCP:', metrics.lcp)
}
```



Próximos Passos

1. **Service Worker:** Implementar cache estratégico

2. **Critical CSS**: Extrair CSS crítico inline
3. **Resource Hints**: Expandir preloads baseados em navegação
4. **Image Optimization**: Implementar WebP/AVIF automático
5. **Bundle Analysis**: Monitoramento contínuo de tamanho

Configurações Adicionais

Bundle Analyzer

Para visualizar o tamanho dos bundles:

```
npm run build:analyze
```

Lighthouse CI

Para integração contínua de performance:

```
npm run perf:lighthouse
```

Métricas de Performance

Monitore as métricas em tempo real usando os hooks fornecidos em `usePerformance.ts`.

Nota: Todas as otimizações foram implementadas seguindo as melhores práticas do Next.js 15 e React 19, focando em melhorar os Core Web Vitals sem comprometer a funcionalidade.